Compositional Dynamic Depth in Neural Networks

A Symmetry-Theoretic Foundation and Experimental Programme

Giulio Ruffini
Neuroelectrics, Starlab – Barcelona, Spain
giulio.ruffini@neuroelectrics.com

Technical Note – Draft, March 2026

Abstract

Standard deep networks apply every layer uniformly to every input. We argue that *depth should be dynamic and input-dependent*, controlled by the compositional complexity of the data-generating process. Building on the Lie-pseudogroup framework of compositional symmetry [Ruffini et al., 2025b, Ruffini, 2025a], we show that the hierarchical flag $G = H_0 \supset H_1 \supset \cdots \supset H_L$ of a generative model naturally prescribes which layers an agent needs to engage for a given input: layers corresponding to inactive symmetry scales should be gated off. We formulate a *symmetry-gated residual network* in which multiplicative gates have precise geometric meaning—they measure the presence of unresolved structure at each scale of the pseudogroup hierarchy. We then lay out a concrete experimental programme using Blender-generated datasets with known compositional structure to validate the theory.

1 Motivation: Why Dynamic Depth?

In a standard L-layer residual network the update rule is

$$h_{\ell+1} = h_\ell + \Delta_\ell(h_\ell), \qquad \ell = 0, 1, \ldots, L-1, \tag{1}$$

where Δ_ℓ is the residual block at layer ℓ. Every input traverses every layer with equal weight. Yet the compositional complexity of natural data varies enormously: a static scene with fixed illumination exercises far fewer generative degrees of freedom than an articulated body under changing viewpoint and lighting.

Several lines of work address this mismatch: Adaptive Computation Time [Graves, 2016], Universal Transformers [Dehghani et al., 2019], Mixture-of-Depths [Raposo et al., 2024], and LayerSkip [Elhoushi et al., 2024]. All share the engineering goal of saving compute by skipping layers. What is missing is a *principled theory that predicts when and why depth should be reduced*—one that connects the gate pattern to the structure of the data-generating process.

This note proposes that the Lie-pseudogroup framework of compositional symmetry [Ruffini et al., 2025b, Ruffini, 2025a] provides exactly such a theory. The central claim is:

Thesis. The effective depth a network needs for a given input is determined by the number of active levels in the compositional factorisation of the data-generating symmetry group. Layers aligned with inactive symmetry scales carry near-zero residuals and should be gated off. Dynamic depth is not an architectural trick—it is a consequence of compositional structure in the world.

2 Theoretical Framework

We briefly recall the essential apparatus; full details are in Ruffini [2025a] and Ruffini et al. [2025b].

2.1 Lie-Pseudogroup Generative Models

Definition 1 (Generative model)**.** *A generative model is a smooth map $f : \mathcal{C} \to \mathbb{R}^X$ from an M-dimensional configuration manifold ($M \ll X$) to observations. It is a* Lie generative model *if a Lie pseudogroup G acts on $\mathcal{C}$ and $\mathbb{R}^X$ such that $f(c) = f(\gamma \cdot c_0) = \gamma \cdot I_0$ for $\gamma \in G$ and a reference observation I_0.*

Near the identity, elements of G factor as

$$\gamma = \gamma^{(L)}\gamma^{(L-1)} \cdots \gamma^{(1)}, \qquad \gamma^{(k)} \in A_k, \tag{2}$$

where each A_k is a local sub-pseudogroup generated by "level-k" infinitesimal symmetries. This is the formal expression of compositional nesting: complex deformations are built by composing a few primitive moves close to the identity.

2.2 The Symmetry Flag and Reduced Manifolds

Choosing the factorisation order in (2) induces a *flag of sub-pseudogroups*,

$$G = H_0 \supset H_1 \supset \cdots \supset H_L, \qquad H_k := A_{k+1} \cdots A_L, \tag{3}$$

obtained by successively omitting generators. Each step $H_{k-1} \to H_k$ declares a new set of symmetry directions "already explained" at coarser scale. The quotients $H_{k-1}/H_k \simeq A_k$ collect the *residual directions* that remain to be explained at level k.

Passing to orbit spaces produces nested *reduced manifolds*:

$$\mathcal{M}_0 \supset \mathcal{M}_1 := \mathcal{M}_0/H_1 \supset \cdots \supset \mathcal{M}_L := \mathcal{M}_{L-1}/H_L, \qquad \dim \mathcal{M}_{k+1} < \dim \mathcal{M}_k. \tag{4}$$

Motion within $\mathcal{M}_k$ is driven only by residual generators in H_k.

2.3 Dynamic Depth from the Flag

Now consider an input stream $I_{\theta(t)}$ in which only generators at levels $1, \ldots, K$ are active (i.e. $\gamma^{(k)} \approx e$ for $k > K$). Then by construction:

- The data lie on $\mathcal{M}_K$, the reduced manifold obtained by quotienting out the inactive levels.
- At any level $k > K$, the residual $\varepsilon_k \in H_{k-1}/H_k$ is near the identity: there is no unexplained structure at that scale.
- A world-tracking agent that mirrors this hierarchy should produce near-zero updates at layers $k > K$.

This is the geometric origin of dynamic depth: **the effective depth equals the number of active compositional levels in the generative process.**

3 Symmetry-Gated Residual Architecture

We now translate the theory into a concrete neural architecture.

3.1 General Formulation

Replace the standard residual rule (1) with

$$\boxed{h_{\ell+1} = h_\ell + \alpha_\ell(x, h_\ell)\, \Delta_\ell(h_\ell), \qquad \alpha_\ell \in [0, 1].} \tag{5}$$

When $\alpha_\ell \approx 0$ the layer is effectively skipped; when $\alpha_\ell \approx 1$ it contributes fully. This is a soft, differentiable version of dynamic depth.

Geometric interpretation. In the symmetry-theoretic picture, layer ℓ corresponds to level $k(\ell)$ of the flag. The gate α_ℓ measures whether the input has unresolved structure in the quotient H_{k-1}/H_k:

$$\alpha_\ell \approx \|\text{residual component in } \mathrm{Lie}(A_{k(\ell)})\|. \tag{6}$$

This is not a free scalar learned by gradient descent alone—it has a *predicted* relationship to the compositional complexity of the input.

3.2 Gate Design Choices

Granularity. The gate can operate at different granularities:

Granularity	Gate shape	Interpretation
Per-sequence	$\alpha_\ell(x) \in \mathbb{R}$	One depth profile per input
Per-token	$\alpha_{\ell,t}(h_{\ell,t}) \in \mathbb{R}$	Token-wise depth
Per-channel/head	$\alpha_{\ell,t,c} \in \mathbb{R}^d$	Fine-grained routing

For the Blender experiments (Section 4), per-sequence gating suffices because the generative hierarchy is a property of the scene, not of individual pixels. For language, per-token gating is more natural.

Soft vs. hard gates. Soft gates (sigmoid) are easy to train and allow fractional contribution. Hard gates (top-k, straight-through) save FLOPs but require tricks for gradient flow. We recommend starting with soft gates and adding a *compute penalty* to encourage sparsity:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \sum_{\ell=1}^{L} \alpha_\ell, \tag{7}$$

where $\lambda > 0$ controls the depth–accuracy trade-off.

Budgeted variant. For a fixed compute budget C, one can impose $\sum_\ell \alpha_\ell = C$ using a softmax or Sinkhorn normalisation over layers. This is closer in spirit to Mixture-of-Depths.

3.3 Concrete Gate Module

A minimal implementation:

$$\alpha_\ell = \sigma\big(w_\ell^\top \,\mathrm{Pool}(h_\ell) + b_\ell\big), \tag{8}$$

where Pool is global average pooling (for images) or `[CLS]` extraction (for transformers), $w_\ell \in \mathbb{R}^d$, $b_\ell \in \mathbb{R}$, and σ is the sigmoid. This adds $d+1$ parameters per layer—negligible overhead.

For token-wise gating:

$$\alpha_{\ell,t} = \sigma(W_\ell\, h_{\ell,t} + b_\ell), \qquad W_\ell \in \mathbb{R}^{1\times d}. \tag{9}$$

3.4 Relationship to Predictive Coding

The predictive hierarchy of Ruffini [2025a] prescribes a richer per-layer algorithm than a simple gate:

$$\text{predict} \;\to\; \text{compare} \;\to\; \text{update locally via } P_k \;\to\; \text{canonicalise} \;\to\; \text{coarse-grain via } \mathcal{C}_{k\to k+1} \;\to\; \text{pass up.} \tag{10}$$

Here $P_k : \mathfrak{g} \to \mathfrak{h}_k$ is the orthogonal projector onto $\mathrm{Lie}(H_k)$, and $Q_k = I - P_k$ extracts the quotient directions. Each level absorbs what it can explain ($P_k\eta_k$) and forwards only the unresolved remainder ($Q_k\eta_k$) after canonicalisation and coarse-graining.

The simple gate α_ℓ in (5) is a *scalar summary* of this richer process: $\alpha_\ell \approx \|Q_{k(\ell)}\eta_{k(\ell)}\|$. Building architectures that implement the full predict–compare–canonicalise–coarse-grain loop at each level is a natural next step (see Section 6).

4 Experimental Programme

The key advantage of this framework is that we can *predict* gate patterns from the known generative hierarchy, and then test those predictions empirically. This requires a dataset where the compositional structure is fully controlled.

4.1 The Blender Cat Rig as a Testbed

Following Appendix C of Ruffini [2025a], we use a production-quality Blender rig (e.g. the cat from the *Flow* project, or any rigged character) whose configuration space decomposes as

$$\mathcal{C} \simeq \mathcal{C}_{\text{camera}} \times \mathcal{C}_{\text{body}} \times \mathcal{C}_{\text{spine}} \times \mathcal{C}_{\text{limbs}} \times \mathcal{C}_{\text{face}} \times \mathcal{C}_{\text{fur}} \times \mathcal{C}_{\text{light}} \times \mathcal{C}_{\text{env}}. \tag{11}$$

The eight levels correspond to sub-pseudogroups $A_1, \ldots, A_8$ (Table 1).

Table 1: Lie-pseudogroup hierarchy for the Blender cat rig.

Level k	Generators A_k	Blender construct	$\dim A_k$
1	Camera $SE(3)$ + lens	`camera.matrix_world`, focal length	7
2	Global body/root $SE(3)$	`pose.bones["root"]`	6
3	Torso/spine chain (PoE)	Spine bones, FK/IK	$\sim$12
4	Limbs/paws/tail (PoE)	Limb bones, IK solvers	$\sim$24
5	Facial morphology	Shape keys (blendshapes)	$\sim$20
6	Fur/appearance	Material nodes, groom PCs	$\sim$6
7	Illumination	Light transforms, SH coeffs	$\sim$12
8	Environment/camera jitter	World rotation, exposure	$\sim$4

4.2 Dataset Generation Pipeline

We propose generating the dataset entirely through Blender's Python API (`bpy`), which avoids any need to "patch" Blender and can be run headless on a compute cluster.

Step 1: Acquire or build a rigged character. Options in decreasing order of realism:

1. Download a production-ready rigged cat (e.g. from BlenderKit, Sketchfab, or TurboSquid—many are free under CC licences). Look for rigs with armature, shape keys, and shader-based materials.
2. Use the free "Cat" asset from the Blender demo files.
3. Build a simpler parametric model (e.g. an articulated stick figure) for a first proof-of-concept.

Step 2: Python script for controlled variation. A Blender Python script that:

1. Defines the A_k parameter ranges (joint angle limits, shape-key ranges, material parameter ranges, light position/colour ranges, camera orbit).
2. For each sample, draws $\theta^{(k)} \sim \text{Uniform}(A_k)$ for the *active* levels and holds inactive levels at their default (rest) values.
3. Writes pose bone transforms via `pose.bones[name].rotation_quaternion`, shape key values via `key_blocks[name].value`, material parameters via node socket values, light transforms via `light.matrix_world`, and camera via `camera.matrix_world` and `camera.data.lens`.
4. Renders to an image buffer using Cycles or Eevee and saves the image together with a metadata JSON recording all parameter values and which levels were active.

Step 3: Construct controlled datasets. For each subset $S \subseteq \{1, \ldots, 8\}$ of active levels, generate N images with only those levels varying. The key conditions are:

Condition name	Active levels
Static	$\emptyset$ (single image, repeated)
CameraOnly	$\{1\}$
CameraBody	$\{1, 2\}$
CameraBodySpine	$\{1, 2, 3\}$
FullPose	$\{1, 2, 3, 4\}$
PoseFace	$\{1, 2, 3, 4, 5\}$
AllGeometric	$\{1, \ldots, 5\}$
AllPlusAppearance	$\{1, \ldots, 7\}$
Everything	$\{1, \ldots, 8\}$

Step 4: Run headless. All rendering can be launched from the command line:

```
blender --background scene.blend \
        --python generate_dataset.py -- \
        --condition CameraBody --n_samples 5000
```

No GUI needed; this runs on any Linux node with a GPU.

4.3 Training Protocol

Architecture. Take a standard ResNet-50 (or Vision Transformer) and insert a gate module (8) at every residual block. Train on the Everything condition with the gated loss (7).

Baselines.

1. *Standard ResNet-50*: no gates (fixed depth).
2. *Random gates*: α_ℓ sampled uniformly, to control for the regularisation effect of the penalty.
3. *ACT-style halting*: a single halting probability rather than per-layer gates.

4.4 Measurements and Predictions

The theory makes the following **testable predictions**:

1. **Gate–complexity alignment.** When the trained model is evaluated on the controlled conditions, the gate vector $(\alpha_1, \ldots, \alpha_L)$ should exhibit a clear transition: gates corresponding to layers aligned with inactive levels should be significantly smaller than those aligned with active levels.

   Metric: For each condition with K active levels, compute the mean gate activation for "active" vs. "inactive" layer groups. The theory predicts a monotone relationship: more active levels $\Rightarrow$ more layers engaged.

2. **Depth–complexity curve.** Define the *effective depth* $D_{\text{eff}}(x) := \sum_\ell \alpha_\ell(x)$. The theory predicts D_{eff} increases with the number of active generative levels:

$$D_{\text{eff}}(\text{CameraOnly}) < D_{\text{eff}}(\text{CameraBody}) < \cdots < D_{\text{eff}}(\text{Everything}). \tag{12}$$

3. **Residual structure.** The residual norms $\|\Delta_\ell(h_\ell)\|$ at gated-off layers should be small not merely because the gate suppresses them, but because the *pre-gate* residual itself carries little energy at that scale. This distinguishes our account from pure regularisation artifacts.

4. **Generalisation to novel compositions.** Train on conditions up to FullPose ($K = 4$), then evaluate on PoseFace ($K = 5$) without retraining. The theory predicts that gate 5 (face) will activate to accommodate the new level, demonstrating compositional generalisation of the depth profile.

5 Relationship to Prior Work

Adaptive Computation Time (ACT) [Graves, 2016]. ACT learns a scalar halting probability per step in a recurrent (shared-weight) architecture. Our framework differs in three ways: (i) layers are *distinct* (different A_k at each level), so dynamic depth means selecting which *compositions of different functions* to apply; (ii) gates have geometric meaning tied to the symmetry hierarchy; (iii) the prediction of gate patterns is derived from the generative model, not learned from scratch.

Mixture-of-Depths [Raposo et al., 2024]. MoD routes a subset of tokens through each layer via top-k selection. This is per-token, hard-gated depth allocation. Our soft per-layer gating is complementary; the budgeted variant of (7) with a softmax over layers converges to a similar mechanism.

Neural ODEs [Chen et al., 2018]. Neural ODEs make depth continuous; adaptive step-size solvers implement a form of dynamic depth. Our gate magnitude α_ℓ plays the role of a step size in a discretised ODE. The connection to ODE stability analysis (Lyapunov functions, ISS bounds) is made explicit in Ruffini [2025a], Equation (5).

Deep Equilibrium Models [Bai et al., 2019]. DEQs take dynamic depth to the limit of infinite (shared) layers and find a fixed point. Our framework suggests that the fixed-point view is appropriate when the compositional hierarchy is *recursive* (shared block), but *compositional* hierarchies with distinct levels require distinct layers—making gated residual networks the natural architecture.

6 Future Directions

1. **Full predictive-coding architecture.** Replace the scalar gate with the full predict–compare–project–canonicalise–coarse-grain loop of (10). Each layer would contain: (a) a predictor $\hat{I}_k = \hat{\gamma}_k \cdot I_0$, (b) a comparator computing $\delta I_k = I - \hat{I}_k$, (c) a linear residual fit $\eta_k \approx \sum_a \eta_k^a V_a^{(k)}$, (d) a projection $P_k \eta_k$ (local update) and $Q_k \eta_k$ (upward message), (e) a coarse-graining operator $\mathcal{C}_{k\to k+1}$. This is architecturally richer than a gated ResNet and directly implements the pseudogroup hierarchy.

2. **Symmetry discovery.** Rather than prescribing the flag, learn the generators $\{T_k\}$ and the factorisation from data using methods such as LieGG [Moskalev et al., 2022] or the approach of Hu et al. [2024]. The learned flag would then *predict* the gate pattern.

3. **Language and transformers.** Syntactic/semantic compositionality in language provides a natural (if less controlled) hierarchy. Per-token gating in a transformer could reveal whether depth allocation correlates with measures of linguistic compositional complexity (syntactic depth, number of variable bindings, logical nesting).

4. **Brain signals.** EEG/neural signals exhibit varying temporal complexity. A dynamic-depth model for neural decoding where the model allocates more compositional processing to complex signal epochs would connect to both the neuroscience of processing difficulty and to KT's account of structured experience.

7 Summary

The standard dynamic-depth literature (ACT, MoD, LayerSkip) approaches the problem from an engineering direction: save compute by skipping layers. We approach it from the opposite direction: the Lie-pseudogroup framework of compositional symmetry *predicts* when and why depth should be dynamic.

The experimental programme proposed here—using Blender rigs with fully controlled compositional structure—provides a direct test: train a gated-depth network, then verify that the learned gate pattern matches the symmetry-predicted depth profile. If it does, we have the first empirical demonstration that dynamic depth in neural networks reflects the compositional structure of the data-generating process.

The contribution is not another gated-depth mechanism. It is a *theory that explains when and why dynamic depth emerges*, grounded in the geometry of Lie pseudogroups, together with a concrete plan to test it.

References

S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. In *NeurIPS*, 2019.

R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.

M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser. Universal transformers. In *ICLR*, 2019.

M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosseini, R. Rabbat, and C.-J. Hsieh. LayerSkip: Enabling early exit inference and self-speculative decoding. *arXiv:2404.16710*, 2024.

A. Graves. Adaptive computation time for recurrent neural networks. *arXiv:1603.08983*, 2016.

L. Hu, Y. Li, and Z. Lin. Symmetry discovery for different data types. *arXiv:2410.09841*, 2024.

A. Moskalev, A. Sepliarskaia, I. Sosnovik, and A. Smeulders. LieGG: Studying learned Lie group generators. *arXiv*, 2022.

D. Raposo, S. Ritter, B. Richards, T. Lillicrap, P. Conway Humphreys, and A. Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv:2404.02258*, 2024.

G. Ruffini. Compositional symmetry as compression: Lie-pseudogroup structure in algorithmic agents. *arXiv:2510.10586*, 2025.

G. Ruffini, F. Castaldo, and J. Vohryzek. Structured dynamics in the algorithmic agent. *Entropy*, 27(1):90, 2025. 10.3390/e27010090.